

# Effectiveness of Machine Learning Methods Using Regression

To what extent are supervised machine learning algorithms effective in predicting the income of adults?

Computer Science



3933 words

# Table of Contents:

|                                                 |    |
|-------------------------------------------------|----|
| Contents: .....                                 | 1  |
| Introduction: .....                             | 2  |
| Types of Algorithms:.....                       | 4  |
| Regression and Classification:.....             | 5  |
| Supervised Learning:.....                       | 7  |
| Support Vector Regression:.....                 | 7  |
| Logistic Regression:.....                       | 8  |
| K-Nearest Neighbors:.....                       | 9  |
| Decision Trees:.....                            | 10 |
| Experiment:.....                                | 11 |
| Background:.....                                | 11 |
| Language of Choice:.....                        | 11 |
| Evaluation of Choice:.....                      | 11 |
| Methodology:.....                               | 12 |
| Hypothesis:.....                                | 12 |
| Quantification of Data:.....                    | 13 |
| Preparing the Algorithms and Accuracy Test..... | 16 |
| Analysis:.....                                  | 18 |
| Limitations:.....                               | 19 |
| Conclusion:.....                                | 19 |
| Further Research:.....                          | 20 |
| References:.....                                | 21 |
| Appendix:.....                                  | 23 |

## Introduction:

Machine learning is a crucial aspect of the software in artificial intelligence since it allows a computer to deduct a pattern or a rule from given data and then process similar data to recognize patterns in new data sets. Since its creation, machine learning has come from being a tool in the “laboratory to a practical technology in widespread commercial use” (Mitchell, 2015, p. 255). Especially in the last 20 years, machine learning has become essential in its applications, from chatbots to search engines. Its applications are seen everywhere on the internet as a way of automation through pattern recognition. For example, suppose we want the computer to recognize a set of handwritten letters. In that case, the computer is given a rough range of ways a letter can be written using pixels, which it incorporates into the data. Now, when the computer processes handwriting, it identifies which pixelated letter is the closest to the one stored in the database and matches it to the corresponding letter. By repeating this multiple times and allowing the computer to output the correct letters correctly, it can reliably output digital text from handwritten text. While this idea initially seems simplistic, the complexities behind how the computer figures out and tunes this relationship between the input and the output are studied through machine learning.

This machine learning process has four components: “information from the training set, answers, the algorithm, and the relationship between the input and the answers” (Agin, 2017, p. 2). Training datasets are initially fed to the algorithm, which outputs the answers. Using the example of handwriting detection, the training dataset is the collection of handwritten letters. The algorithm is the code to decide how the computer compares and recognizes letters. Then, by tuning the relationship between the input and the answers, the computer can identify letters consistently. Finally, the answer is outputted based on the tuning and the algorithm based on the path the input traveled. These components are necessary for any machine learning algorithm to output accurate responses. In this research, I will

explain the different types of machine learning algorithms, specifically regression algorithms, and run an experiment to determine their accuracy compared to each other.

## Types of Algorithms:

There are three different types of algorithms: supervised learning, unsupervised learning, and reinforcement learning. Before moving on, the term “feature” describes the independent variables, or the data being fed to the computer, and the term “label” describes the dependent variable, or the data that the computer is supposed to output based on the features of the data. Therefore, when data is unlabelled, the machine has no human-determined categories to output. Supervised learning uses these features and labels to aid in tasks for more consistent results. In contrast, unsupervised learning chooses its own goals, which are naturally unlabelled, creating more inconsistencies. The format of my data and its goal is best represented through supervised learning because of my ability to control the output and reliably measure the accuracy. Also, reinforcement learning is not applicable because it approaches machine learning by focusing on “goal-directed learning from interaction” (Barto, 2014, p. 1). In other words, it allows the machine to decide its goal, focusing on learning from experience, like someone learning how to drive would. Although I will only detail supervised learning, “the goal of the learner is always the same—minimize the total loss coming from making predictions” (Hunter & Freitag, 2019, p. 319).

## Regression and Classification:

Regression and classification are the two most commonly used techniques within supervised machine learning algorithms. Regression requires the machine to numeric data and then create labels for output. Using this data, it creates a prediction using the data according to the labels. On the other hand, classification groups data based on discrete values, splitting up the data evenly. For example, given a selection of numbers, the human would classify each number as a label. One label would be 0, and all the handwritten numbers would be compiled for the computer to use for classification.



*Figure 1: Towards Data Science.*

<https://towardsdatascience.com/part-5-training-the-network-to-read-handwritten-digits-c2288f1a2de3>

In comparison, regression would take various factors and have labels for the results of these factors, such as the cost of a stock based on the previous prices and changes. While classification is used to predict groups and where new data would fall in these predetermined groups, regression uses a variety

of factors to find a prediction based on the relationships between the factors. The different types of machine learning algorithms use different forms of mathematical regression, which results in calculating the output differently. Regression is commonly used to predict house prices, stock market prices, and so much more.

Using the adult data set with sixteen identifying factors, the machine learning algorithms will use regression to predict whether or not an adult earns 50,000 USD per year based on age, occupation, marital status, and other factors. There can only be two outputs: they earn 50 thousand USD, or they do not earn 50 thousand USD. The significance of this data set is that it demonstrates correlations between factors in adults to find their income. Using machine learning techniques is significant because it demonstrates how they can identify people based on other factors. Such ideas are helpful in similar identification technologies, such as identifying medical diseases based on external factors that the patient can provide. This research can demonstrate correlations in workplace biases, such as whether gender or race correlate. However, outside of the data set, it also reveals the accuracy of many popular regression models in an extensive set. Through this, it would reveal the strengths and limitations of each model and their use cases.

## Supervised Learning:

A wide variety of regression algorithms can apply to this data. For the most extensive research, I will take some of the most popular regression algorithms: support vector machine, logistic regression, decision trees, and k-nearest neighbor. These algorithms are all able to complete the task and provide a demonstration of their accuracy.

### Support Vector Regression:

The support vector regression algorithm executes regression tasks but specializes in data sets with higher dimensions than the number of samples. This algorithm is an extension of the Support Vector Machine, which uses a similar method for classification. This algorithm rose to popularity through handwriting recognition when it reached a 3.5% error rate on recognition, close to the human error of 2.5% (Moguerza, 2006, p.336). The algorithm maps the points onto a plane, and then, based on the relationship between the data, a hyperplane or a boundary is placed to divide the data. In this example (*see Figure 2*), the points in the middle are bounded away from the rest of the points by a hyperplane, creating two sets of labeled data. In doing so, it provides results based on tuning and can make accurate cuts to divide the data. Despite its efficiency for large data sets, a downside of support vector machines is that processing large data sets takes much time.

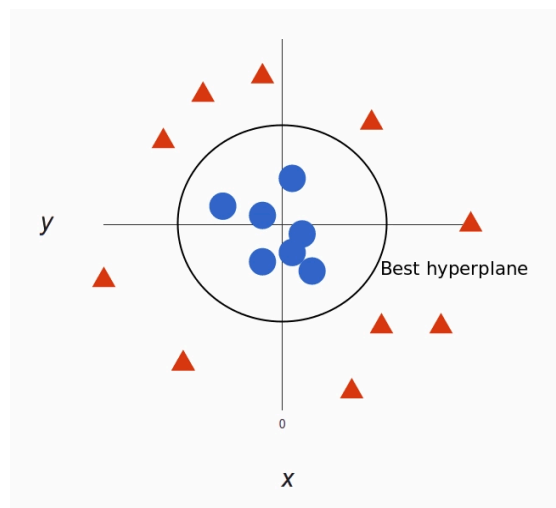
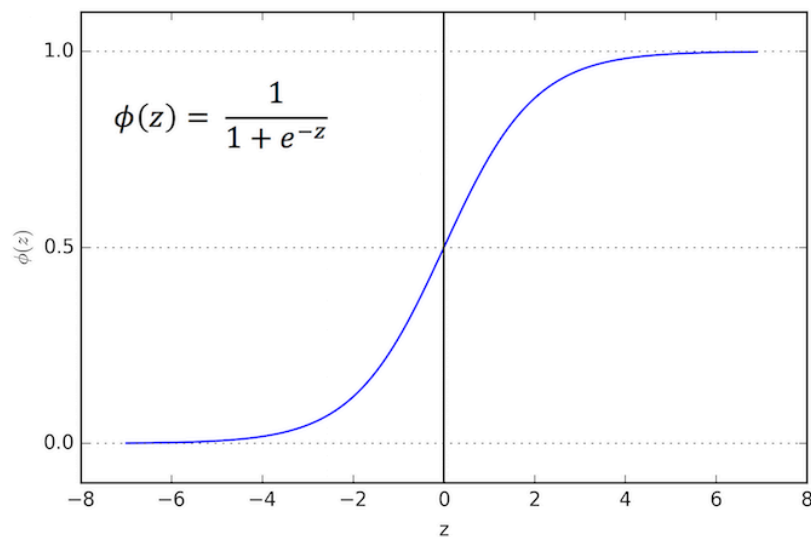


Figure 2: 2-Dimensional Hyperplane. [monkeylearn.com/blog/classification-algorithms/](http://monkeylearn.com/blog/classification-algorithms/)



## Logistic Regression:

Another commonly used supervised learning algorithm is logistic regression. The sigmoid function below (see Figure 3) charts a graph to plot the data points. They are plotted on a graph using the relationship between the data points. Based on whether the data point is closer to 0 or 1 on the y-axis, it outputs 0 or 1 for each corresponding number. For example, if a person in my experiment had many characteristics associated with a higher salary, the data point for the person would be plotted more correctly since these qualities are associated with a positive relationship. In doing so, it splits up the participants based on these predictions. This form of logistic regression is binary because it can only output two things, which limits its usage. However, despite its simplicity, it is convenient, as it is very good at forecasting events due to the reliability of the sigmoid function. Since the data points can be easily split up and plotted on the algorithm, this algorithm is also very easy to train, as it does not require much training time to be reliable.



*Figure 3: Sigmoid Graph Used for Plotting Points.*

[towardsdatascience.com/11-most-common-machine-learning-algorithms-explained-in-a-nutshell-cc6e98df93be](https://towardsdatascience.com/11-most-common-machine-learning-algorithms-explained-in-a-nutshell-cc6e98df93be).

## K-Nearest Neighbors:

Another supervised learning algorithm is k-nearest neighbors, which uses the “k” amount of data points to put themselves into classes. The “k” in k-nearest neighbors is a variable for the amount of neighbors per class. For example, given 12 points, the algorithm would split the points into three classes given k was 4. The algorithm first calculates the distance between the new and existing samples in the training dataset. The samples are then sorted by distance to the new sample, and the k-nearest neighbors are selected. The average of these k-nearest neighbors is then used to predict the new sample. K-nearest neighbors are a simple machine learning algorithm, but it struggles with large data sets, as it has to divide them all and calculate them individually manually. Additionally, it is a non-parametric method, meaning it makes no assumptions about the data distribution (e.g., fitting the bell curve). This makes the algorithm robust for diverse data sets that do not fit normal distribution.

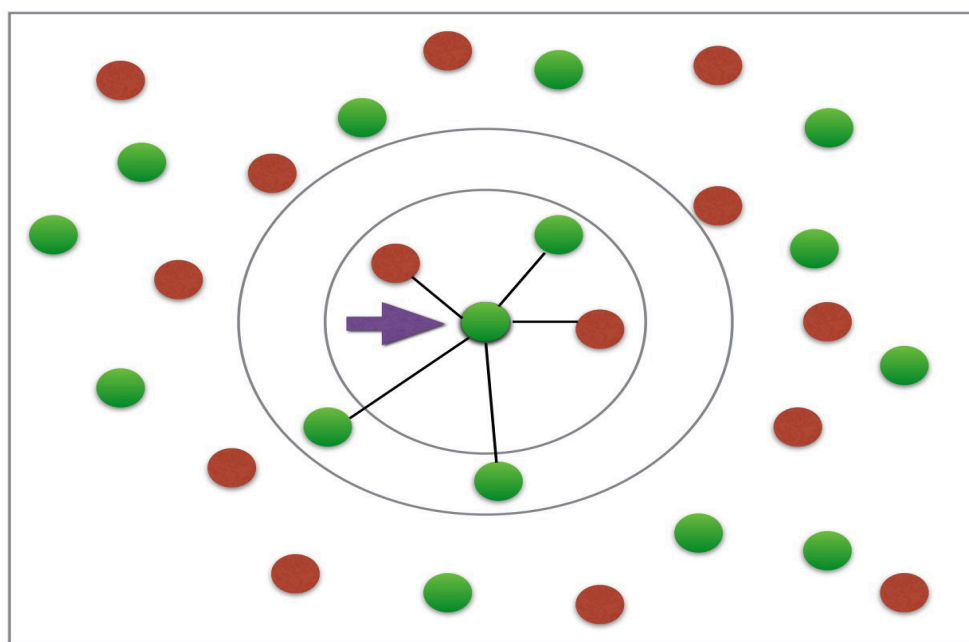


Figure 4: Figure with Circles Around Associations in Dataset

[towardsdatascience.com/11-most-common-machine-learning-algorithms-explained-in-a-nutshell-cc6e98df93be](https://towardsdatascience.com/11-most-common-machine-learning-algorithms-explained-in-a-nutshell-cc6e98df93be).

## Decision Trees:

Decision trees are a popular supervised learning algorithm for classification and regression. The algorithm creates a tree-like decision model based on the data provided. The tree is constructed by making a series of decisions based on the attributes of the data, with each branch of the tree representing a possible outcome of the decision. This process continues until a final prediction can be made. Figure 5 shows the starting root node, essentially the first yes or no question the computer asks. Then, it continues to split until the leaf nodes, or the final answers to the decision tree, after all the factors have been considered. Decision trees are a simple and effective method for making predictions, but they can be prone to overfitting.

Additionally, decision trees can be difficult to interpret, making it challenging to explain the predictions made by the model. If interpreted correctly, however, since the decision tree is visual, this can be useful in situations where it is essential to explain the reasoning behind the predictions, such as in medicine or finance. Decision trees can also handle numerical and categorical data, making them a versatile algorithm.

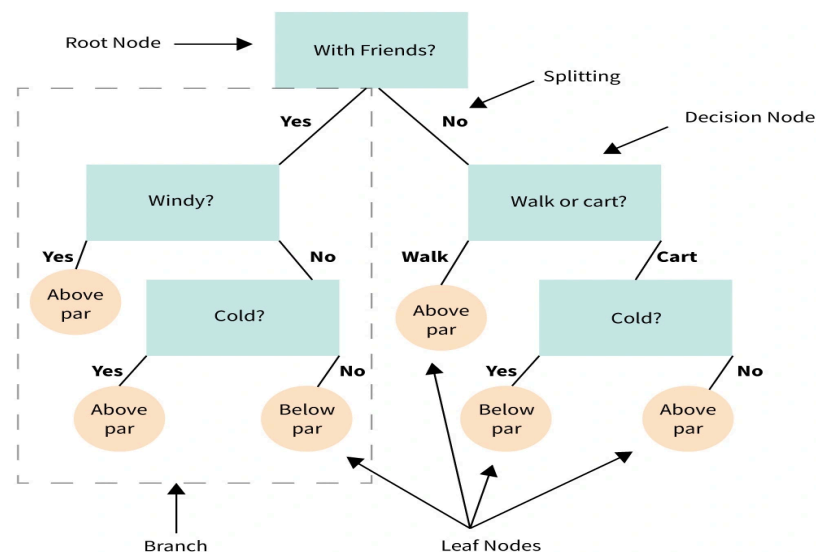


Figure 5: Golfing Decision Tree. [machinelearningmastery.com](http://machinelearningmastery.com)

## **Experiment:**

### **Background:**

My experiment will take the machine learning algorithms I covered and test their prediction accuracy. They require the same data set to level the playing field between the algorithms. The data I used was found in the UC Irvine Machine Learning database. Based on the basic adult database donated in 1996, 32561 adults were surveyed on sixteen factors: age, work class, final weight, education, education number, marital status, occupation, relationship, sex, race, capital gain, capital loss, hours per week, native country, and income. This data is both recorded categorically and through integers and was recorded in the 1994 census. In the experiment, it is essential to convert the categorical data into integers to meet the needs of the machine learning algorithms, which require numerical values.

### **Language of Choice:**

For this experiment, the language of choice was Python. It is optimal because it is the simplest to understand and is the most commonly used for machine learning. Python also has an extensive library of machine learning tools and algorithms, which create more reusable and readable code. In this case, the Kaggle tool is also helpful because it creates an all-in-one environment where the data and Python code are all stored. It also helps debug the code, simplifying solving errors. It is also helpful because it directly creates visualizations that supplement the code as you write it, showing both progress and valuable data.

### **Evaluation of choice:**

The evaluation is an accuracy score built into the Python database for regression. It gauges the accuracy of the regression based on a scale of 0 to 1, with 1 being perfect results. It calculates the percentage of correctly identified labels. It takes the correctly predicted divided by the total number of predictions. For example, if the algorithm correctly predicted 75 out of 100 results, it would have an accuracy score of 0.75. The accuracy score of each algorithm is its average result in this test, showing how often the algorithm got the correct results based

on the data provided. This is effective because it provides a tool to compare the accuracy of each algorithm for the data set evenly. Due to the same data set being used for the algorithms, the accuracy score will stay similar between trials, so conducting multiple tests on the same data set is unnecessary.

## **Methodology:**

In the experiment, chosen data and a database had to be provided to access the machine learning algorithms. After supplying Kaggle with the data and converting it into a comma-separated values (CSV) file used for data tables, the databases chosen were required to run commands necessary for machine learning. In order to execute basic Python functions for data science, the pandas, numpy, and matplotlib.pyplot libraries are commonly used to read data, execute mathematical operations, and plot visualizations. The sklearn database contains many subsets containing machine learning algorithms and tools used for them, which make the code more readable.

## **Hypothesis:**

Logistic regression is the most effective, followed by decision trees, k-nearest neighbor, and support vector machine, which is the least accurate.

## Quantification of data:

In order to quantify all the data, all the categorical data had to be converted into numerical values for the computer to read. The testing data needed a metric to determine the significance of the input. Therefore, by converting the salaries into numbers, the computer can understand how all the other factors lead to a resulting value by mapping a salary under 50 thousand to 1 and over 50 thousand dollars to 0

```
salary_map={' <=50K':1, ' >50K':0}
df['salary']=df['salary'].map(salary_map).astype(int)
```

*Figure 6: Code Converting Salary Level into Booleans. kaggle.com*

Next, all the other categorical factors had to be quantified. Starting with the sex factor, a similar mapping was done with males and females to 0 and 1.

```
df['sex'] = df['sex'].map({' Male':1, ' Female':0}).astype(int)
```

*Figure 7: Conversion of Sex to Booleans kaggle.com*

Before continuing, a problem found throughout the dataset was the absence of information for different factors such as county, workclass, and occupation. Without information in these categories, the algorithms could not calculate the data. Therefore, removing the rows where the data cannot be processed is essential. Using the dropna command, any row missing data in any of the columns gets removed from the data set, cleaning the data.

```
df['country'] = df['country'].replace(' ?', np.nan)
df['workclass'] = df['workclass'].replace(' ?', np.nan)
df['occupation'] = df['occupation'].replace(' ?', np.nan)
df.dropna(how='any', inplace=True)
```

*Figure 8: Removal of Empty Data. kaggle.com*

After cleaning the data, the individual factors can be sorted similarly to the salary and placed in categories. Afterward, they can be mapped into boolean values by categorizing their inputs as different numbers. For this factor and other factors mapped to another number, an average mean of each country's significance in the data set (how much the computer considers this factor in the final output) is stored and accessed when the data checks for the location of the mapping.

```
for dataset in data:
    dataset.loc[dataset['country'] != 'United-States', 'country'] = 'Non-US'
    dataset.loc[dataset['country'] == 'United-States', 'country'] = 'US'
df['country'] = df['country'].map({'US':1, 'Non-US':0}).astype(int)
```

*Figure 9: Mapping Countries to Booleans. kaggle.com*

Next, as marital status and relationship are related, we can use them together to categorize each other. For marital status, the two options are single and in a couple, so the different marital statuses that result in adults being single: Divorced, Married-spouse absent, Never-married, Separated, and Widowed are all categorized as the same, while adults being in a married couple: Married-AF-spouse and Married-civ-Spouse. The impact of each statistic in Figure 10 demonstrates the individual impact of marital status, with a number closer to 1 being its correlation with salary.

|                |                | salary   |
|----------------|----------------|----------|
| relationship   | marital-status |          |
| Husband        | Couple         | 0.544331 |
| Not-in-family  | Couple         | 0.714286 |
|                | Single         | 0.893802 |
| Other-relative | Couple         | 0.857143 |
|                | Single         | 0.976623 |
| Own-child      | Couple         | 0.821429 |
|                | Single         | 0.988818 |
| Unmarried      | Single         | 0.933686 |
| Wife           | Couple         | 0.506401 |

*Figure 10: Correlations Between Marital Status and Salary using Logistic Regression. kaggle.com*

Afterward, instead of words, the different relationships and marital statuses must be converted into numbers. As the other factors, they will be mapped to different numbers.

```
df['marital-status'] = df['marital-status'].map({'Couple':0, 'Single':1})

rel_map = {' Unmarried':0, ' Wife':1, ' Husband':2, ' Not-in-family':3, ' Own-child':4, ' Other-relative':5}

df['relationship'] = df['relationship'].map(rel_map)
```

*Figure 11: Mapping Marital Status to Booleans. kaggle.com*

The next factor to be categorized into numerical values is race, where each given race is mapped to a number.

```
race_map={' White':0, ' Amer-Indian-Eskimo':1, ' Asian-Pac-Islander':2, ' Black':3, ' Other':4}

df['race'] = df['race'].map(race_map)
```

*Figure 12: Mapping Race to Booleans. kaggle.com*

The following factors dealt with were ones related to jobs. As both work class and occupation could be summarized by employment type, they each fell under a category. However, there were too many different occupations to list as different mappings, so they were sorted under employment type. Each work class was assigned to an employment type, covering all the options and summarizing them into four categories. These were chosen because they were the most broad classes that were not repetitive of the other data. After they were assigned, they were all mapped to numbers as other factors had been.

These choices left three redundant categories that would not affect the data but slow down the process, so they could be removed from the data tables using a drop command. These categories were the workclass, which had already been summarized using the employment type label, and the occupation label, which was too diverse and partially summarized by the employment type label. The education label was also removed because of the already existing education measured, counting the education level in numbers.

```
df.drop(labels=['workclass', 'education', 'occupation'], axis=1, inplace=True)
```

*Figure 13: Removing Redundant Labels. kaggle.com*



The last two factors to consider are capital gain and capital loss. These factors are important because they alter how much money an individual makes in their salary. Although capital gain and loss were uncommon in the data set, they still showed significant impact to be considered in the algorithms. Therefore, due to its scarcity, any capital gain or loss was marked to be noticed by the algorithm and mapped to booleans.

```
df.loc[(df['capital-gain'] > 0), 'capital-gain'] = 1
df.loc[(df['capital-gain'] == 0), 'capital-gain'] = 0

df.loc[(df['capital-loss'] > 0), 'capital-loss'] = 1
df.loc[(df['capital-loss'] == 0), 'capital-loss'] = 0
```

*Figure 14: Capital Gain and Loss Mapped to Booleans. kaggle.com*

## Preparing the Algorithms and Accuracy Test:

Finally, we can begin training the algorithms on the formatted data after all the data is converted to integers. The data must be split between the training, validation, and testing data. The computer uses the training data to prepare the model with part of the data. The validation data tests the model's accuracy and tunes how the computer will predict the data. Finally, the test data uses the remaining data to test how accurately the algorithm executes the regression task. The ratio used is 50 percent test data, 20 percent validation data, and 30 percent testing data. While this is no specific way to know the “best” ratio, after testing multiple ratios, this ratio allows for the algorithms to accurately distribute the data and train the algorithm enough to be accurate.

```
from sklearn.model_selection import train_test_split

X= df.drop(['salary'],axis=1)
y=df['salary']

split_size=0.3

#Creation of Train and Test dataset
X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=split_size,random_state=22)

#Creation of Train and validation dataset
X_train, X_val, y_train, y_val = train_test_split(X_train,y_train,test_size=0.2,random_state=5)
```

*Figure 15: Distribution of Data. kaggle.com*

Afterward, we import all of the machine learning algorithms from the sklearn database. They are prepared to execute the algorithms with a prepared code.

```
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC

models = []
names = ['SVM', 'KNN', 'LR', 'DecisionTreeClassifier']

models.append((KNeighborsClassifier()))
models.append((LogisticRegression()))
models.append((DecisionTreeClassifier()))
models.append((SVC()))
```

*Figure 16: Importing Algorithms. kaggle.com*

The final step is to import the accuracy score I will use. Then, for each machine learning algorithm, the process is run across the data set given, and it returns an accuracy score of the predicted data by the algorithm versus the actual algorithm.

```
from sklearn import model_selection
from sklearn.metrics import accuracy_score

kfold = model_selection.KFold(n_splits=5, random_state=7)

for i in range(0, len(models)):
    cv_result = model_selection.cross_val_score(models[i], X_train, y_train, cv=kfold, scoring='accuracy')
    score=models[i].fit(X_train, y_train)
    prediction = models[i].predict(X_val)
    acc_score = accuracy_score(y_val, prediction)
    print ('-'*40)
    print ('{0}: {1}'.format(names[i], acc_score))
```

*Figure 17: Loop to Print Out Accuracy Score For Each Algorithm. kaggle.com*

## Analysis:

As a result of the experiment, each machine learning algorithm outputted an accuracy score between 0 and 1, representing the percentage of accurate predictions. The support vector machine algorithm had an accuracy test score of 0.7094. This algorithm was the least accurate out of all the algorithms, likely because it is difficult to tune the algorithm properly; making even a small mistake cost the accuracy highly. While it still performed relatively accurately in the test, it demonstrated that it was ineffective for regression compared to the other algorithms. The K-Nearest Neighbors algorithm had an accuracy test score of 0.7492 and was sorted in the middle of the algorithms. Since there were many factors, the K-Nearest Neighbors algorithm was not inhibited by outliers, as it had many factors to rely on. However, this algorithm struggled with the dataset's size, which may have limited its performance. The Logistic Regression algorithm had an accuracy score of 0.7801, the highest of all the algorithms. Using the logistic regression algorithm was simple yet surprisingly effective. The sigmoid function, which is used many times, fits the data well and is additionally very reliable for different data types, showing its versatility and strength compared to other regression algorithms. The Decision Tree algorithm had an accuracy test score of 0.7509, but its accuracy varied throughout different trials. The accuracy score fluctuated between 0.7450 and 0.7550, likely due to the fact that small changes in the training data greatly influenced the output of the function. However, it was still accurate to a strong extent, similar to the K-Nearest Neighbors algorithm, due to its effectiveness in finding patterns in data.

## **Limitations:**

Although the verdict of the experiment is rather obvious based on the data, some limitations may draw a different conclusion. The data used may have been specifically skewed towards one algorithm or another. With more varying types of data, a more overarching conclusion could be established, but with only one piece, we cannot conclude on the overall effectiveness of the algorithms. This data set may not represent all society, although it measures many people. Another limitation was splitting the data while testing, meaning that different amounts of training, validation, and testing could have resulted in different results. Although, this would likely not have led to a different conclusion.

## **Conclusion:**

Therefore, I will accept my hypothesis, as each algorithm performed to my expectations. The experiment's results showcased that it is possible to predict the salary of any person with enough given factors to a degree of certainty. A simple test like this can have a bigger effect on the world, showing that machine algorithms can draw important information from various factors. This demonstrates the element of cybersecurity within machine learning algorithms, as they can find information based on common patterns among people. With more developed algorithms with large amounts of data, results could become more conclusive based on similar factors. However, since there is no guaranteed answer due to other factors that cannot be measured, there is no need to worry, as no algorithm can be completely confident about any result. This experiment is still significant because it demonstrates how the linear regression model is especially effective in using regression to forecast the outcome of this data, while the support vector machine is inaccurate in predicting the outcome of this data and demonstrates how common factors can be used to predict personal information.

## **Further Research:**

In future research, it may be effective to run more algorithms, as the sample set of 4 algorithms is not representative of all supervised algorithms that can be used in regression. These algorithms could provide more data and be either more accurate or less accurate than those already studied. Some may be more qualified in predicting the income of adults based on given factors. Another aspect for future research is more supervised learning types, as regression does not represent all supervised learning. Therefore, if classification were used, it could provide more concrete results. The final topic for further research could be exploring other factors that affect the salary of an adult. This could include employment time, specified location, or familial wealth. These other factors could change the outcome of the test, and with more factors and more versatility among different types of classification and algorithm types, our conclusion could be more extensive than the data set chosen.

## References:

- Agin, Warren E. "A Simple Guide to Machine Learning." *Business Law Today*, 2017, pp. 1–5. *JSTOR*, <https://www.jstor.org/stable/90003559>. Accessed 3 Nov. 2022.
- Becker, Barry and Kohavi, Ronny. (1996). Adult. UCI Machine Learning Repository. <https://doi.org/10.24432/C5XW20>.
- Chase, Hunter, and James Freitag. "MODEL THEORY AND MACHINE LEARNING." *The Bulletin of Symbolic Logic*, vol. 25, no. 3, 2019, pp. 319-32. *JSTOR*, [www.jstor.org/stable/26788523](https://www.jstor.org/stable/26788523). Accessed 9 Feb. 2024.
- DeMaris, Alfred. "A Tutorial in Logistic Regression." *Journal of Marriage and Family*, vol. 57, no. 4, 1995, pp. 956-68. *JSTOR*, <https://doi.org/10.2307/353415>. Accessed 9 Feb. 2024.
- Different Types of Logistic Regression*. [www.mastersindatascience.org/learning/machine-learning-algorithms/logistic-regression/](http://www.mastersindatascience.org/learning/machine-learning-algorithms/logistic-regression/).
- esri*. [pro.arcgis.com/en/pro-app/3.1/tool-reference/geoai/how-decision-tree-classification-and-regression-works.htm](http://pro.arcgis.com/en/pro-app/3.1/tool-reference/geoai/how-decision-tree-classification-and-regression-works.htm).
- Ghosh, Anil K. "On Nearest Neighbor Classification Using Adaptive Choice of K." *Journal of Computational and Graphical Statistics*, vol. 16, no. 2, 2007, pp. 482-502. *JSTOR*, [www.jstor.org/stable/27594253](https://www.jstor.org/stable/27594253). Accessed 9 Feb. 2024.
- GitHub*, [github.com/bipin2295/ML\\_income\\_prediction\\_on\\_census\\_UCI\\_data/blob/master/Consensus.py](https://github.com/bipin2295/ML_income_prediction_on_census_UCI_data/blob/master/Consensus.py). Accessed 14 Feb. 2024.
- Goldberger, Arthur S. "Reverse Regression and Salary Discrimination." *The Journal of Human Resources*, vol. 19, no. 3, 1984, pp. 293-318. *JSTOR*, <https://doi.org/10.2307/145875>. Accessed 9 Feb. 2024.

*javatpoint*. [www.javatpoint.com/accuracy\\_score-in-sklearn](http://www.javatpoint.com/accuracy_score-in-sklearn).

*javatpoint*. [www.javatpoint.com/unsupervised-machine-learning](http://www.javatpoint.com/unsupervised-machine-learning).

Jordan, M. I., and T. M. Mitchell. "Machine Learning: Trends, Perspectives, and Prospects."

*Science*, vol. 349, no. 6245, 2015, pp. 255–60. *JSTOR*, [http://www.jstor.org/stable/](http://www.jstor.org/stable/24748571)

24748571. Accessed 3 Nov. 2022.

*Kaggle*. [www.kaggle.com/hopertime/notebook/edit](http://www.kaggle.com/hopertime/notebook/edit).

Moguerza, Javier M., and Alberto Muñoz. "Support Vector Machines with Applications." *Statistical*

*Science*, vol. 21, no. 3, 2006, pp. 322-36. *JSTOR*, [www.jstor.org/stable/27645765](http://www.jstor.org/stable/27645765). Accessed 9

Feb. 2024.

*Monkey Learn*. [monkeylearn.com/blog/classification-algorithms/](http://monkeylearn.com/blog/classification-algorithms/).

*Reinforcement Learning: An Introduction*. Second ed., MIT Press. *Stanford*, MIT Press,

[web.stanford.edu/class/psych209/Readings/SuttonBartoIPRLBook2ndEd.pdf](http://web.stanford.edu/class/psych209/Readings/SuttonBartoIPRLBook2ndEd.pdf). Accessed 7 Nov. 2022.

*Toward Data Science*. [towardsdatascience.com/](http://towardsdatascience.com/)

11-most-common-machine-learning-algorithms-explained-in-a-nutshell-cc6e98df93be. Accessed 7

Nov. 2022.

## Appendix:

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from pandas.plotting import scatter_matrix
# Loading csv file in dataframe
df = pd.read_csv("../input/adult.csv",1,"")
data = [df]
print (df.head())
# Convert salary to integer
salary_map={' <=50K':1,' >50K':0}
df['salary']=df['salary'].map(salary_map).astype(int)
print (df.head(10))
# convert sex into integer
df['sex'] = df['sex'].map({' Male':1,' Female':0}).astype(int)
print (df.head(10))
print ((" "*40))
print (df.info())
# Find correlation between columns
def plot_correlation(df, size=15):
    corr= df.corr()
    fig, ax =plt.subplots(figsize=(size,size))
    ax.matshow(corr)
    plt.xticks(range(len(corr.columns)),corr.columns)
    plt.yticks(range(len(corr.columns)),corr.columns)
    plt.show()
plot_correlation(df)
plot_correlation(df)
plot_correlation(df)
# Categorise in US and Non-US candidates

```



```

print (df[['country','salary']].groupby(['country']).mean())
# Drop empty value marked as '?'
print (df.shape)
df['country'] = df['country'].replace(' ?',np.nan)
df['workclass'] = df['workclass'].replace(' ?',np.nan)
df['occupation'] = df['occupation'].replace(' ?',np.nan)
df.dropna(how='any',inplace=True)
print (df.shape)
print (df.head(10))
for dataset in data:
    dataset.loc[dataset['country'] != ' United-States', 'country'] = 'Non-US'
    dataset.loc[dataset['country'] == ' United-States', 'country'] = 'US'
df.head(10)
# Convert country in integer
df['country'] = df['country'].map({'US':1,'Non-US':0}).astype(int)
df.head(10)
# Categorise marital-status into single and couple
df['marital-status'] = df['marital-status'].replace([' Divorced',' Married-spouse-absent',' Never-married','
Separated',' Widowed'],'Single')
df['marital-status'] = df['marital-status'].replace([' Married-AF-spouse',' Married-civ-spouse'],'Couple')
df.head(10)
df[['marital-status','salary']].groupby(['marital-status']).mean()
df[['marital-status','relationship','salary']].groupby(['marital-status','relationship']).mean()
df[['marital-status','relationship','salary']].groupby(['relationship','marital-status']).mean()
df['marital-status'] = df['marital-status'].map({'Couple':0,'Single':1})
df.head(10)
rel_map = {' Unmarried':0,' Wife':1,' Husband':2,' Not-in-family':3,' Own-child':4,' Other-relative':5}
df['relationship'] = df['relationship'].map(rel_map)
df.head(10)
# Analyse *race*
df[['race','salary']].groupby('race').mean()

```

```

race_map={' White':0,' Amer-Indian-Eskimo':1,' Asian-Pac-Islander':2,' Black':3,' Other':4}
df['race']= df['race'].map(race_map)
df.head(10)
df[['occupation','salary']].groupby(['occupation']).mean()
df[['workclass','salary']].groupby(['workclass']).mean()
def f(x):
    if x['workclass'] == ' Federal-gov' or x['workclass']== ' Local-gov' or x['workclass']==' State-gov':
return 'govt'
    elif x['workclass'] == ' Private':return 'private'
    elif x['workclass'] == ' Self-emp-inc' or x['workclass'] == ' Self-emp-not-inc': return 'self_employed'
    else: return 'without_pay'
df['employment_type']=df.apply(f, axis=1)
df.head(10)
df[['employment_type','salary']].groupby(['employment_type']).mean()
employment_map = {'govt':0,'private':1,'self_employed':2,'without_pay':3}
df['employment_type'] = df['employment_type'].map(employment_map)
df.head(10)
df[['education','salary']].groupby(['education']).mean()
df.drop(labels=['workclass','education','occupation'],axis=1,inplace=True)
df.head(10)
x= df['education-num']
plt.hist(x,bins=None,density=True,normed=None,histtype='bar')
plt.show()
x=df['capital-gain']
plt.hist(x,bins=None,normed=None)
plt.show()
df.loc[(df['capital-gain'] > 0),'capital-gain'] = 1
df.loc[(df['capital-gain'] == 0 ,'capital-gain')]= 0
df.head(25)
x=df['capital-loss']
plt.hist(x,bins=None)

```

```

plt.show()
df.loc[(df['capital-loss'] > 0), 'capital-loss'] = 1
df.loc[(df['capital-loss'] == 0, 'capital-loss')] = 0
df.head(10)
df['age'].count()
# 50% training data, 20% validation data, 30% test data
from sklearn.model_selection import train_test_split
X= df.drop(['salary'],axis=1)
y=df['salary']
split_size=0.3
#Creation of Train and Test dataset
X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=split_size,random_state=22)
#Creation of Train and validation dataset
X_train, X_val, y_train, y_val = train_test_split(X_train,y_train,test_size=0.2,random_state=5)
print ("Train dataset: {0} {1}".format(X_train.shape, y_train.shape))
print ("Validation dataset: {0} {1}".format(X_val.shape, y_val.shape))
print ("Test dataset: {0} {1}".format(X_test.shape, y_test.shape))
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
models = []
names = ['SVM','KNN','LR','DecisionTreeClassifier']
models.append((KNeighborsClassifier()))
models.append((LogisticRegression()))
models.append((DecisionTreeClassifier()))
models.append((SVC()))
print (models)
from sklearn import model_selection
from sklearn.metrics import accuracy_score
kfold = model_selection.KFold(n_splits=5,random_state=7)

```

```
for i in range(0,len(models)):
    cv_result = model_selection.cross_val_score(models[i],X_train,y_train,cv=kfold,scoring='accuracy')
    score=models[i].fit(X_train,y_train)
    prediction = models[i].predict(X_val)
    acc_score = accuracy_score(y_val,prediction)
    print ('-'*40)
    print ('{0}: {1}'.format(names[i],acc_score))
```